

A2 – Case Study: Stage 4 Tutorial Activities

Object-Oriented Design Decisions

Activity 1 – Encapsulation Review:

Class	Protected State/Invariant	Public Operations
Patient	Patient_id, name, contact_details should not be changed directly without validation. A patient needs to always have valid identifying information.	Update_details(), view_details()
Practitioner	Practitioner_id, name, availability should remain consistent and only be updated through class methods.	Update_availability(), view_schedule()
Appointment	Appointment_id, patient, practitioner, data_time, status should remain valid, status should follow allowed transitions and appointments should not create practitioner double-bookings.	Create(), update_status(), cancel()

Activity 2 – Composition or Inheritance?:

Relationship	Decision	Reason
Appointment and Patient	Composition	An appointment is linked to a patient, but an appointment is not a type of patient.
Appointment and Practitioner	Composition	An appointment involves a practitioner, but it is not a specialised practitioner.
Doctor and Practitioner	Inheritance	A doctor could be considered a specialised type of practitioner, so inheritance would be appropriate if this distinction were required.
Clinic and Appointment	Composition	A clinic may contain or manage appointments,

		but an appointment is not a type of clinic.
--	--	---

Activity 3 – Responsibility Allocation:

Who decides whether SCHEDULED can become CANCELLED?

The appointment class should decide, the status belongs to Appointment therefore Appointment should contain the rule that determines whether a status transition is valid. Keeping the business logic close to the data it affects.

Who validates a patient name?

The patient class should validate the patient name, because the name is part of the patient state therefore patient should make sure the name is valid before it is stored or updated.

Should Appointment execute SQL? Why?

No, appointment should contain appointment-related business logic, not database access code. Database access should be handled by a separate persistence/repository layer if one is later introduced.

Should the UI decide whether a status transition is legal?

No, the UI should collect input and display results, but the appointment class should decide whether a status change is valid. Otherwise, the same business rule could be duplicated across different screens or interfaces.

Activity 4 – AI Code Critique:

AI Design Problem	Why is this a problem?	Correction
Public Status Mutation	Any code could change the appointment status directly and bypass business rules.	Make status protected and only change it through methods such as cancel() or update_status().
SQL Inside	This mixes appointment business logic with database logic, making the class harder to test and maintain.	Move database access into a separate persistence or repository component.
Notification Manager Dependency	Notifications are not part of the confirmed SmartCare requirements, so this adds unsupported complexity.	Remove the dependency unless notifications are later confirmed by the client.

Appointment inherits from PatientRecord	An appointment is not a type of PatientRecord, so inheritance is logically incorrect.	Use an association between Appointment and Patient instead.
Appointment has too many responsibilities	The class is handling status rules, database access and notifications instead of focusing on appointment behaviour.	Keep appointment responsible only for appointment data and business rules.

Exit Question:

Why can code be object-oriented syntactically but still have poor object-oriented design?

Code can use classes, objects and methods but still have poor object-oriented design if responsibilities are placed in the wrong classes, data is not properly protected, or classes are too dependent on each other. Good object-oriented design requires clear responsibilities, encapsulation and logical relationships between classes, not just object-oriented syntax.

SmartCare v0.4 – Domain Implementation Workbook

1. UML-to-Code Trace:

UML Element	Python Element	Implemented	Notes
Patient Class	Class patient	Yes	Matches stage 3 model
Patient_ID	Self.patient_id	Yes	Stores unique patient identifier
Name	Self.name	Yes	Stores patient name
Contact_Details	Self.contact_details	Yes	Stores patient contact information
Practitioner Class	Class practitioner	Yes	Matches stage 3 model
Practitioner_id	Self.practitioner_id	Yes	Stores practitioner identifier
Avaliability	Self.avaliability	Yes	Stores practitioner avaliability

2. Domain Invariants:

Class	Invariant	How Protected
Patient	Patient information should remain valid and consistent.	Details are updated through class methods rather than direct uncontrolled changes.
Patient	A patient should have valid identifying information.	Validation is performed before saving or updating details.
Practitioner	Practitioner availability should remain consistent.	Availability is changed through <i>update_avaliability()</i> .
Appointment	An appointment must be associated with a patient and practitioner.	Patient and practitioner references are stored inside the appointment object.
Appointment	Appointment status should only change through valid operations.	Status is protected as <i>_status</i> and changed through methods.
Appointment	A practitioner should not be double-booked for the same time.	Booking logic checks practitioner availability before an appointment is created.

3. Composition/Inheritance Decisions:

Relationship	Decision	Rationale
Appointment and Patient	Association	An appointment relates to a patient but is not a type of patient.
Appointment and Practitioner	Association	An appointment relates to a practitioner but is not a type of practitioner.
Doctor and Practitioner	Inheritance	A doctor could be considered a specialised type of practitioner if required.
Clinic and Appointment	Association	A clinic may manage appointments, but an appointment is not a type of clinic.
Appointment and PatientRecord	No inheritance	An appointment is not specialised PatientRecord, so inheritance would be incorrect.

4. AI Pair-Programming Record

AI Contribution	Conforms?	Decision	Reason	Verification
Suggested protecting appointment status	Yes	Accepted	Supports encapsulation and prevents invalid direct changes.	Checked against Stage 4 requirement that status mutation should not be public
Suggested SQL inside <i>cancel()</i>	No	Rejected	Mixes business logic with database access and makes testing harder.	Compared with responsibility allocation principles from Stage 4.
Suggested <i>NotificationManager</i>	No	Rejected	Notifications are not part of the confirmed SmartCare requirements.	Checked against case study and functional requirements.
Suggested Appointment inherit from <i>PatientRecord</i>	No	Rejected	An appointment is not a type of <i>PatientRecord</i> .	Checked using the 'is-a' inheritance rule.

Case Study Stage 4 Lab Activities:

Implementing the SmartCare Domain Layer

A – Revisit Approved UML:

Responsibilities:

Patient Class: Patient information needs to remain valid and consistent and are updated through class methods rather than direct uncontrolled changes.

Patient ID: Ensures the each confirmed patient class has a valid identification which is performed before saving or updating details.

Practitioner Class: Practitioner availability remains constant with the available dates and times, *update_availability* changes the availability.

Attributes:

Patient: *patient_id, name, contact_details*

Practitioner: *practitioner_id, name, speciality*

Appointment: *appointment_id, patient, practitioner, date_time, status*

Relationships:

Patient-to-Appointment: One-to-Many

Practitioner-to-Appointment: One-to-many

Patient-to-Practitioner: No direct relationship